

Student Dropout Predictor

```
In [649... import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from xgboost import XGBClassifier
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, cl
```

```
In [650... df = pd.read_csv("Machine_Learning_Task_Database.csv")
df.head()
```

```
Out[650... Student_ID  Attendance  Favourite_Color  Previous_Score  Random_Code  Age  Study_Ho

0      S149795      82.3      Green      44.0      563097.0  31.0
1      S134468      94.1      Green      NaN      620165.0  46.0
2      S136603      64.7%      Yellow      41.0      458806.0  19.0
3      S119995      68.9      White      32.0      250069.0  52.0
4      NaN      49.3      Red      47.0      573746.0  18.0
```

```
In [651... df.drop([
    "Student_ID",
    "Random_Code",
    "Lucky_Number",
    "Random_Value",
    "Random_Flag",
    "Random_Category",
    "Random_Number_2",
    "Random_Date_Code",
    "Random_Score",
    "Random_Bucket"
], axis=1, inplace=True)
df.head()
```

```
Out[651... Attendance  Favourite_Color  Previous_Score  Age  Study_Hours  Gender  Absences  Edu

0      82.3      Green      44.0  31.0      1.3      F      17.0
1      94.1      Green      NaN  46.0      6.1      F      22.0
2      64.7%      Yellow      41.0  19.0      1.4      F      27.0  D
3      68.9      White      32.0  52.0      0.7  female      8.0
4      49.3      Red      47.0  18.0      4.4  Female      7.0  Gr
```

```
In [652... inputs = df.drop("Outcome", axis=1)
le_outcome = LabelEncoder()
```

```
df["Outcome"] = le_outcome.fit_transform(df["Outcome"])
targets = df["Outcome"]
```

In [653...

inputs

Out[653...

	Attendance	Favourite_Color	Previous_Score	Age	Study_Hours	Gender	Absences
0	82.3	Green	44.0	31.0	1.3	F	17.0
1	94.1	Green	NaN	46.0	6.1	F	22.0
2	64.7%	Yellow	41.0	19.0	1.4	F	27.0
3	68.9	White	32.0	52.0	0.7	female	8.0
4	49.3	Red	47.0	18.0	4.4	Female	7.0
...	...	...	...	...	...	...	...
49995	83.0	White	89.0	34.0	2.8	M	8.0
49996	76.3	Red	34.0	34.0	5.8h	male	22.0
49997	50.0	Black	91.0	47.0	7.1	M	16.0
49998	61.5	Blue	48.0	56.0	1.9	Male	29.0
49999	NaN	Black	70.0	37.0	1.6	female	0.0

50000 rows × 30 columns



In [654...

targets.head()

Out[654...

```
0    1
1    0
2    1
3    1
4    0
Name: Outcome, dtype: int64
```

In [655...

```
cleaning_map_for_Gender = {
    'f': 'Female',
    'female': 'Female',
    'm': 'Male',
    'male': 'Male'
}
cleaning_map_for_Academic_Warning = {
    'yes': 'Yes',
    'no': 'No'
}
cleaning_map_for_Financial_Aid = {
    'yes': 'Yes',
    'no': 'No'
}
def clean_inputs(inputs, clean_map, column_name):
    inputs[column_name] = inputs[column_name].str.lower()
```

```

inputs[column_name] = inputs[column_name].map(clean_map)
inputs[column_name] = inputs[column_name].replace(np.nan, inputs[column_name].mode().top(1).mode[0])
le = LabelEncoder()
inputs[column_name] = le.fit_transform(inputs[column_name])
clean_list = ["Gender", cleaning_map_for_Gender], ["Academic_Warning", cleaning_map_for_Academic_Warning]
for column, clean_map in clean_list:
    clean_inputs(inputs, clean_map, column)

```

In [656...

```

def fill_nan_with_mean(column):
    inputs[column] = inputs[column].fillna(inputs[column].mean()).round(2)

num_list = [
    'Attendance',
    'Previous_Score',
    'Age',
    'Study_Hours',
    'Absences', 'Income',
    'Assignments_Completed',
    'Stress_Level',
    'Previous_Failures',
    'Parent_Support',
    'Financial_Stress',
    'Library_Visits',
    'Commute_Minutes',
    'Mock_Test_Score',
    'Mentor_Meetings',
    'Sleep_Hours',
    'Extracurricular_Hours',
    'Registration_Quarter',
    'Assignment_Submission_Rate'
]

inputs['Attendance'] = inputs['Attendance'].astype(str).str.replace('%', '', regex=True)
inputs['Study_Hours'] = inputs['Study_Hours'].astype(str).str.replace('hrs', '', regex=True)
inputs['Income'] = inputs['Income'].astype(str).str.replace('$', '', regex=True)
has_k = inputs['Income'].str.contains('k', na=False)
inputs['Income'] = inputs['Income'].str.replace('k', '', regex=True).astype(float)
inputs['Income'] = np.where(has_k, inputs['Income'] * 1000, inputs['Income'])
for column in num_list:
    fill_nan_with_mean(column)
inputs

```

Out[656...

	Attendance	Favourite_Color	Previous_Score	Age	Study_Hours	Gender	Absences
0	82.30	Green	44.00	31.0	1.3	0	17.0
1	94.10	Green	65.12	46.0	6.1	0	22.0
2	64.70	Yellow	41.00	19.0	1.4	0	27.0
3	68.90	White	32.00	52.0	0.7	0	8.0
4	49.30	Red	47.00	18.0	4.4	0	7.0
...	...	...	...	...	...	...	...
49995	83.00	White	89.00	34.0	2.8	1	8.0
49996	76.30	Red	34.00	34.0	5.8	1	22.0
49997	50.00	Black	91.00	47.0	7.1	1	16.0
49998	61.50	Blue	48.00	56.0	1.9	1	29.0
49999	67.44	Black	70.00	37.0	1.6	0	0.0

50000 rows × 30 columns



In [657...

```
def fill_nan_with_mode(column):
    inputs[column] = inputs[column].fillna(inputs[column].mode()[0])
categorical_list = [
    'Favourite_Color',
    'Education',
    'Music_Preference',
    'City',
    'Pet_Ownership',
    'Internet_Quality',
    'Device',
]
for column in categorical_list:
    fill_nan_with_mode(column)
encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded = encoder.fit_transform(inputs[categorical_list])
encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out(categorical_list),
    index=inputs.index
)
inputs = inputs.drop(columns=categorical_list)
inputs = pd.concat([inputs, encoded_df], axis=1)
inputs
```

Out[657...

	Attendance	Previous_Score	Age	Study_Hours	Gender	Absences	Income	Assign
0	82.30	44.00	31.0	1.3	0	17.0	91133.00	
1	94.10	65.12	46.0	6.1	0	22.0	71100.00	
2	64.70	41.00	19.0	1.4	0	27.0	8200.00	
3	68.90	32.00	52.0	0.7	0	8.0	41080.00	
4	49.30	47.00	18.0	4.4	0	7.0	10069.00	
...	...	...	...	...	...	...	...	...
49995	83.00	89.00	34.0	2.8	1	8.0	63834.03	
49996	76.30	34.00	34.0	5.8	1	22.0	69381.00	
49997	50.00	91.00	47.0	7.1	1	16.0	63834.03	
49998	61.50	48.00	56.0	1.9	1	29.0	80374.00	
49999	67.44	70.00	37.0	1.6	0	0.0	39643.00	

50000 rows × 51 columns

In [658... `X_train, X_test, y_train, y_test = train_test_split(inputs, targets, test_size=0.2,`

```

In [659... model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    random_state=42
)
model.fit(X_train, y_train)

```

Out[659...

▼ XGBClassifier ⓘ ?

- Parameters
- Fitted attributes



```

In [660... accuracy = accuracy_score(y_test, model.predict(X_test))
accuracy

```

Out[660... 0.6601